

Exp 1 (IPC)

Client:

```
import socket
def client_program():
    client = socket.socket() #makes the socket
    client.connect(('localhost', 65432)) # bind and listen for server,
connect for client
    while True:
        msg = input("You: ")
        client.send(msg.encode()) # send to talk
        if msg.lower() == "exit":
            print("Client closing.")
            break
        reply = client.recv(1024).decode() # recv to listen
        if reply.lower() == "exit":
            print("Server left.")
            break
        print("Server:", reply)
    client.close()
client_program()
```

Server:

```
import socket
def server_program():
    server = socket.socket()
    server.bind(('localhost', 65432))
    server.listen(1) # listen(1) = "Allow 1 client to wait in line
while I process connections."
    print("Server waiting...")
    conn, addr = server.accept()
    print("Connected to", addr)
    while True:
        msg = conn.recv(1024).decode()
        if msg.lower() == "exit":
            print("Client left.")
            break
        print("Client:", msg)
        reply = input("You: ")
        conn.send(reply.encode())
        if reply.lower() == "exit":
            print("Server closing.")
            break
```

```
conn.close()
server.close()
server_program()
```

Exp 2 (Group Communication)

Client:

```
# group_client.py
import socket
import threading

def receive(sock):
    while True:
        try:
            msg = sock.recv(1024).decode()
            if not msg:
                break
            print(msg)
        except:
            break

def start_client():
    client = socket.socket()
    client.connect(('127.0.0.1', 55555))
    print(client.recv(1024).decode(), end='') # "Enter your name:"
    name = input()
    client.send(name.encode())
    threading.Thread(target=receive, args=(client,),
daemon=True).start()
    while True:
        msg = input()
        client.send(msg.encode())
        if msg.lower() == "exit":
            break
    client.close()
start_client()
```

Server:

```
# group_server.py
import socket
import threading

clients = {}

def broadcast(msg, sender=None):
    for client in list(clients):
        if client != sender:
```

```

        try:
            client.send(msg.encode())
        except:
            client.close()
            del clients[client]

def handle_client(sock):
    sock.send("Enter your name: ".encode())
    name = sock.recv(1024).decode().strip()
    clients[sock] = name
    print(f"{name} joined the chat.")
    broadcast(f"{name} joined the chat.", sock)
    while True:
        try:
            msg = sock.recv(1024).decode()
            if msg.lower() == "exit":
                break
            print(f"{name}: {msg}")
            broadcast(f"{name}: {msg}", sock)
        except:
            break
    print(f"{name} left the chat.")
    broadcast(f"{name} left the chat.", sock)
    sock.close()
    del clients[sock]

def start_server():
    server = socket.socket()
    server.bind(('127.0.0.1', 5555))
    server.listen()
    print("Server started.")
    while True:
        client, _ = server.accept()
        threading.Thread(target=handle_client, args=(client,),
daemon=True).start()
start_server()

```

Exp 3 (Distributed Global Averaging Method)

```
import random
NUM_NODES=5
nodes = [random.randint(1, 100) for _ in range(NUM_NODES)]
print("Initial node values:", nodes)
ROUNDS=8
for r in range(ROUNDS):
    print(f"\n--- Round {r + 1} ---")
    random.shuffle(nodes)

    for i in range(0, len(nodes) - 1, 2):
        avg = (nodes[i] + nodes[i + 1]) / 2
        nodes[i] = avg
        nodes[i + 1] = avg
        print(f"Node {i} and Node {i+1} exchanged values -> New value: {avg}")
    print("Current values:", [round(x, 2) for x in nodes])
    # print("nodes " , *nodes)

final_avg = sum(nodes) / len(nodes)
print("\nFinal approximate global average at all nodes:",
round(final_avg, 2))
```

Exp 4 (Berkley Method)

```
def to_seconds(time_str):
    h, m, s = map(int, time_str.split(":"))
    return h*3600 + m*60 + s

def to_time(seconds):
    h = seconds // 3600
    m = (seconds % 3600) // 60
    s = seconds % 60
    return f"{h:02d}:{m:02d}:{s:02d}"

# same as to time just we have abs function here and sign
def format_offset(offset_seconds):
    sign = "-" if offset_seconds < 0 else "+"
    abs_offset = abs(offset_seconds)
    h = abs_offset // 3600
    m = (abs_offset % 3600) // 60
    s = abs_offset % 60
    return f"{sign}{h:02d}:{m:02d}:{s:02d}"

def berkeley():
    n = int(input("Enter number of processes: "))
    times = []
    for i in range(n):
        t = input(f"Enter time for Process {i} (HH:MM:SS): ")
        times.append(to_seconds(t))
    avg = sum(times) // n
    print("\n--- Synchronized Times ---")
    for i in range(n):
        offset = avg - times[i]
        print(f"Process {i}: {to_time(times[i])} → {to_time(avg)}")
        print(f"Offset : {offset} -> {format_offset(offset)}")
berkeley()
```

Exp 5 (Bully Algorithm)

```
class Process:
    def __init__(self,pid):
        self.pid = pid
        self.alive=True
    def crash(self):
        self.alive=False
    def start_election(self,processes):
        print(f"Process {self.pid} starting election...")
        for proc in processes:
            if proc.pid>self.pid and proc.alive:
                print(f"Process {self.pid} sends election message to {proc.pid}")
                return proc.start_election(processes)
        print(f"Process {self.pid} becomes the new coordinator")
        return self.pid

def bully_algorithm():
    n = int(input('Enter the number of processes:'))
    process=[Process(i) for i in range(n+1)]
    crash_id=int(input('Enter the id of process that crashed'))
    process[crash_id].crash()
    coordinator_id=int(input('Enter id of new coordinator'))
    if not process[coordinator_id].alive:
        print(f"Coordinator {coordinator_id} is down. Starting election..")
        initiator_id=int(input('Enter id of process starting election:'))
        process[initiator_id].start_election(process)
    else:
        print(f"Coordinator {coordinator_id} is still alive")

if __name__ == "__main__":
    bully_algorithm()
```

Exp 6 (Raymond Tree based method)

```
class Node:
    def __init__(self, pid):
        self.pid = pid
        self.has_token = False
        self.request_queue = []
        self.parent = None

    def request_token(self):
        if self.has_token :
            print(f"\nProcess {self.pid} has the token and enters
Critical Section.")
            self.enter_critical_section()
        else:
            print(f"\nProcess {self.pid} requests token from parent
Process {self.parent.pid if self.parent is not None else 'None'}")
            self.parent.receive_request(self)

    def receive_request(self, sender):
        if sender not in self.request_queue:
            self.request_queue.append(sender)
            print(f"Request Queue of Process {self.pid}: {[node.pid for
node in self.request_queue]}")

        if self.has_token and self.request_queue:
            next_node = self.request_queue.pop(0)
            print(f"Process {self.pid} passes token to Process
{next_node.pid}")
            self.has_token = False
            next_node.receive_token(self)

    def receive_token(self, sender):
        self.has_token = True
        self.parent = sender
        self.request_token()

    def enter_critical_section(self):
        print(f"Process {self.pid} is in Critical Section.")
        self.has_token = False # token gone
        if self.request_queue: # if 2 has a req queue
            next_node = self.request_queue.pop(0) # we pop the first
element
```

```

        print(f"Process {self.pid} passes token to Process
{next_node.pid}")
        next_node.receive_token(self) # send it to crititka
    else:
        print(f"Request Queue of Process {self.pid} is empty.
Therefore, Release Critical Section.")

def build_tree(nodes, parent_list):
    for i in range(len(nodes)):
        parent_idx = parent_list[i]
        if parent_idx != -1:
            nodes[i].parent = nodes[parent_idx]

def simulate():
    n = int(input("Enter number of processes: "))
    nodes = [Node(i) for i in range(n)]

    print("Enter parent for each process (-1 for root):")
    parent_list = []
    for i in range(n):
        p = int(input(f"Parent of process {i}: "))
        parent_list.append(p)

    build_tree(nodes, parent_list)

    root = parent_list.index(-1)
    nodes[root].has_token = True
    print(f"\nProcess {root} is the root and initially has the
token.\n")

    while True:
        req = input("Enter process ID to enter CS (or 'exit' to quit):
")
        if req.lower() == "exit":
            break
        pid = int(req) # 2
        nodes[pid].request_token()

if __name__ == "__main__":
    simulate()

```


Exp 7 (Ricart-Agrawala Method)

```
class Process:
    def __init__(self, pid, num_processes):
        self.pid = pid
        self.clock = 0
        self.requesting_cs = False
        self.timestamp = None
        self.reply_count = 0
        self.deferred = []
        self.num_processes = num_processes

    def request_cs(self, all_processes):
        self.clock += 1
        self.timestamp = self.clock
        self.requesting_cs = True
        self.reply_count = 0
        print(f"[Process {self.pid}] requesting CS at time {self.clock}")

        # Send request to all other processes
        for p in all_processes:
            if p.pid != self.pid:
                p.receive_request(self)

        # After collecting all replies
        if self.reply_count == self.num_processes - 1:
            print(f"[Process {self.pid}] has entered the CRITICAL SECTION ✓")
            print(f"[Process {self.pid}] is exiting the CRITICAL SECTION ✗")
            self.release_cs()

    def receive_request(self, sender):
        self.clock += 1
        if not self.requesting_cs or (self.timestamp, self.pid) > (sender.timestamp, sender.pid):
            print(f"[Process {self.pid}] sending reply to Process {sender.pid}")
            sender.receive_reply(self.pid)
        else:
            print(f"[Process {self.pid}] deferring reply to Process {sender.pid}")
            self.deferred.append(sender)

    def receive_reply(self, from_pid):
        print(f"[Process {self.pid}] received reply from Process {from_pid}")
        self.reply_count += 1
```

```

def release_cs(self):
    self.requesting_cs = False
    for p in self.deferred:
        print(f"[Process {self.pid}] sending deferred reply to Process {p.pid}")
        p.receive_reply(self.pid)
    self.deferred.clear()

# Main simulation
num = int(input("Enter the number of processes: "))
processes = [Process(i, num) for i in range(num)]

print("\nYou can now request CS access for any process(es).")
ids = input("Enter process IDs (space-separated) to request CS: ")
ids = list(map(int, ids.split()))

for pid in ids:
    processes[pid].request_cs(processes)

```

Exp 8 (Bankers Algorithm)

```

def main():
    # Input basic info
    processes = int(input("Enter number of processes: "))
    resources = int(input("Enter number of resources: "))

    # Input total resources
    total = list(map(int, input("\nEnter total resources (space separated): ").split()))

    # Input max matrix
    print("\nEnter max resources needed for each process:")
    max_need = [list(map(int, input(f"P{i}: ").split())) for i in range(processes)]

    # Input allocation matrix
    print("\nEnter allocated resources for each process:")
    alloc = [list(map(int, input(f"P{i}: ").split())) for i in range(processes)]

    # Calculate available resources
    allocated = [sum(col) for col in zip(*alloc)]
    available = [total[i] - allocated[i] for i in range(resources)]

```

```

    # Calculate need matrix
    need = [[max_need[i][j] - alloc[i][j] for j in range(resources)]
for i in range(processes)]

    # Safety check
    work = available.copy()
    finish = [False] * processes
    safe_seq = []

    while True:
        found = False
        for i in range(processes):
            if not finish[i] and all(need[i][j] <= work[j] for j in
range(resources)):
                work = [work[j] + alloc[i][j] for j in
range(resources)]
                finish[i] = True
                safe_seq.append(f"P{i}")
                found = True
                break

        if not found:
            break

    # Output results
    print("\nNeed Matrix:")
    for i in range(processes):
        print(f"P{i}:", need[i])

    print("\nAvailable Resources:", *work)

    if all(finish):
        print("\nSystem is SAFE")
        print("Safe sequence:", " → ".join(safe_seq))
    else:
        print("\nSystem is UNSAFE")

if __name__ == "__main__":
    main()

```

Exp 9 (Load Balancing)

```
def load_balancer():
    loads = {0: 4, 1: 3, 2: 3}
    while True:
        print("\nCurrent Load:", loads)
        print("Menu:")
        print("Press 1 to add a process")
        print("Press 2 to remove a process")
        print("Press 3 to add a node")
        print("Press 4 to remove a node")
        print("Press 5 to exit")
        ch = input("Enter your choice:")
        if ch == '1':
            node_id = min.loads, key=loads.get
            loads[node_id] += 1
            print(f"Added process to Node {node_id}")
        elif ch == '2':
            id = int(input("Enter a node id to remove a process from"))
            loads[id] -= 1
            print(f"Process removed from {id}st node")
        elif ch == '3':
            new_id = max.loads.keys() + 1 if loads else 0
            loads[new_id] = 0
            print(f"Added new node {new_id}")
        elif ch == '4':
            node_id = int(input("Enter id to remove:"))
            removed_loads = loads[node_id]
            loads.pop(node_id)
            print(f"Node {node_id} removed. Redistributing {removed_loads}
processes")
            for _ in range(removed_loads):
                key = min.loads, key=loads.get
                loads[key] += 1
        elif ch == '5':
            print('Exiting')
            break
        else:
            print('invalid choice')

if __name__ == "__main__":
    load_balancer()
```

Exp 10 (Multithreading)

```
import threading
import time
lock = threading.Lock()
counter = 0
def workers(name, increments, batch=10000):
    global counter
    print(f"[{time.time()}][{name}] Starting will do {increments} increments")
    for i in range(1, increments+1):
        with lock:
            counter += 1
            if i % batch == 0:
                print(f"[{time.time()}][{name}] Reached {i} increments, counter
={counter}")
    print(f"[{time.time()}][{name}] Finished")
n = int(input('Enter number of threads:'))
incs = int(input('Enter number of increments:'))
threads = []
for i in range(n):
    t = threading.Thread(target=workers, args=(f"Thread-{i+1}",incs))
    threads.append(t)
    t.start()
for t in threads:
    t.join()
print(f"[{time.time()}][main]Final counter = {counter}")
```